

ARCHITECTURAL IMPLEMENTATION SPECIFICATION

CEO DASHBOARD (NO LIMITS COMPANY)

Target Persona: CEO	Backend Stack: Java 17, Spring Boot 3.x, JPA	Database Engine: PostgreSQL 15+
---------------------	--	---------------------------------

Este documento constituye la especificación técnica absoluta para la construcción y despliegue del ecosistema de servicios back-end y la capa relacional de base de datos para el **CEO Dashboard de NO LIMITS COMPANY**. Todos los componentes descritos aquí deben ser dinámicamente expuestos mediante APIs estructuradas.

1. SYSTEM ARCHITECTURE METADATA (BACKEND SCOPE)

- **Arquitectura del Motor:** Basada en microservicios o monolito modular robusto utilizando Java 17 y Spring Boot 3.x.
- **Persistencia de Datos:** Spring Data JPA con mapeo relacional estricto hacia PostgreSQL 15+.
- **Seguridad y Control de Acceso:** Spring Security con soporte para RBAC (Role-Based Access Control) mediante tokens firmados criptográficamente (JWT).
- **Flujo de Datos en Tiempo Real:** Configuración de end-points habilitados para Server-Sent Events (SSE) o WebSockets con el fin de empujar actualizaciones transaccionales inmediatas hacia la capa de presentación.

2. DATABASE SCHEMA ARCHITECTURE (POSTGRESQL)

La estructura base de datos debe asegurar restricciones relacionales estrictas, índices para optimización de consultas analíticas sobre KPIs y registros de auditoría distribuidos. A continuación se detalla el script DDL definitivo:

```
-- 1. USER PROFILE AND AUTH SCOPE
CREATE TABLE users (
    user_id SERIAL PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    role VARCHAR(20) NOT NULL DEFAULT 'CEO',
    avatar_url TEXT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

-- 2. DAILY SUMMARY AND FOCUS

```
CREATE TABLE daily_summaries (  
    summary_id SERIAL PRIMARY KEY,  
    user_id INT REFERENCES users(user_id),  
    summary_date DATE NOT NULL DEFAULT CURRENT_DATE,  
    key_actions TEXT[] NOT NULL,  
    events TEXT[] NOT NULL,  
    focus_text TEXT NOT NULL,  
    CONSTRAINT unique_user_date UNIQUE(user_id, summary_date)  
);
```

-- 3. AGENDA / SCHEDULE

```
CREATE TABLE agenda_events (  
    event_id SERIAL PRIMARY KEY,  
    user_id INT REFERENCES users(user_id),  
    title VARCHAR(150) NOT NULL,  
    description TEXT,  
    start_time TIMESTAMP WITH TIME ZONE NOT NULL,  
    end_time TIMESTAMP WITH TIME ZONE NOT NULL,  
    category VARCHAR(50), -- STRATEGIC, FINANCIAL, OPERATIONAL  
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);
```

-- 4. PRIORITIZED TASKS

```
CREATE TABLE priority_tasks (  
    task_id SERIAL PRIMARY KEY,  
    user_id INT REFERENCES users(user_id),  
    title VARCHAR(200) NOT NULL,  
    progress_percentage INT CHECK (progress_percentage BETWEEN 0 AND 100) DEFAULT 0,  
    status VARCHAR(20) NOT NULL DEFAULT 'PENDING', -- PENDING, IN_PROGRESS, COMPLETED  
    due_date DATE,  
    priority_order INT NOT NULL  
);
```

-- 5. KEY PROJECTS

```
CREATE TABLE key_projects (  
  project_id SERIAL PRIMARY KEY,  
  name VARCHAR(150) NOT NULL,  
  progress_percentage INT CHECK (progress_percentage BETWEEN 0 AND 100) DEFAULT 0,  
  status VARCHAR(30) DEFAULT 'IN_PROGRESS',  
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);
```

-- 6. KEY PERFORMANCE INDICATORS (KPIs) WITH HISTORICAL SERIES

```
CREATE TABLE kpi_metrics (  
  kpi_id SERIAL PRIMARY KEY,  
  metric_code VARCHAR(50) UNIQUE NOT NULL, -- MONTHLY_REVENUE, SALES_GROWTH...  
  metric_name VARCHAR(100) NOT NULL,  
  current_value NUMERIC(15,2) NOT NULL,  
  display_format VARCHAR(20) NOT NULL, -- CURRENCY, PERCENTAGE, NUMBER  
  delta_percentage NUMERIC(5,2) NOT NULL,  
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE kpi_historical_data (  
  history_id SERIAL PRIMARY KEY,  
  kpi_id INT REFERENCES kpi_metrics(kpi_id) ON DELETE CASCADE,  
  datapoint_value NUMERIC(15,2) NOT NULL,  
  recorded_at TIMESTAMP WITH TIME ZONE NOT NULL  
);
```

-- 7. NOTIFICATIONS / URGENT MESSAGES

```
CREATE TABLE urgent_messages (  
  message_id SERIAL PRIMARY KEY,  
  user_id INT REFERENCES users(user_id),  
  source_channel VARCHAR(50) NOT NULL, -- CORREO, SYSTEM, ALARM  
  title VARCHAR(250) NOT NULL,  
  body TEXT,  
  is_read BOOLEAN DEFAULT FALSE,  
  received_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);
```

-- 8. TEAM RECENT ACTIVITY

```
CREATE TABLE team_activities (  
  activity_id SERIAL PRIMARY KEY,  
  team_name VARCHAR(100) NOT NULL,  
  description TEXT NOT NULL,  
  activity_type VARCHAR(50) NOT NULL, -- ACHIEVEMENT, DEPLOYMENT, ALERT  
  logged_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);
```

3. BACKEND SERVICE ARCHITECTURE (SPRING BOOT 3.X)

Implementación idiomática de controladores REST y Data Transfer Objects (DTO) usando encapsulación de código mediante las anotaciones de Lombok.

A. Domain DTO Examples

```
package com.nolimits.dashboard.dto;

import lombok.Data;
import java.math.BigDecimal;
import java.util.List;

@Data
public class KpiDashboardDTO {
    private String metricCode;
    private String metricName;
    private String displayValue;
    private BigDecimal deltaPercentage;
    private List<BigDecimal> sparklineData;
}
```

B. Rest Controller Tier

```
package com.nolimits.dashboard.controller;

import com.nolimits.dashboard.dto.*;
import com.nolimits.dashboard.service.DashboardService;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.time.LocalDate;
import java.util.List;

@RestController
@RequestMapping("/api/v1/dashboard")
@RequiredArgsConstructor
@CrossOrigin(origins = "*")
public class DashboardController {

    private final DashboardService dashboardService;

    @GetMapping("/summary")
    public ResponseEntity<DailySummaryDTO> getDailySummary(@RequestParam Long userId) {
        return ResponseEntity.ok(dashboardService.fetchSummaryByDate(userId, LocalDate.now()));
    }

    @GetMapping("/agenda")
    public ResponseEntity<List<AgendaEventDTO>> getTodayAgenda(@RequestParam Long userId) {
        return ResponseEntity.ok(dashboardService.fetchTodayEvents(userId));
    }

    @GetMapping("/tasks/priority")
    public ResponseEntity<List<PriorityTaskDTO>> getPriorityTasks(@RequestParam Long userId) {
        return ResponseEntity.ok(dashboardService.fetchTopPriorityTasks(userId));
    }

    @GetMapping("/projects/key")
    public ResponseEntity<List<KeyProjectDTO>> getKeyProjects() {
        return ResponseEntity.ok(dashboardService.fetchActiveKeyProjects());
    }

    @GetMapping("/kpis")
    public ResponseEntity<List<KpiDashboardDTO>> getKpiMetrics() {
        return ResponseEntity.ok(dashboardService.calculateLiveKpis());
    }
}
```

4. EXECUTION & DELIVERY REQUIREMENTS

- **No Mock Fallbacks:** Construir definiciones completas de Repositorios JPA vinculando las tablas objetivo a servicios operacionales reales para asegurar flujos mutables correctos.
- **Políticas de Transaccionalidad:** Garantizar el uso adecuado de `@Transactional(readOnly = true)` en operaciones de lectura analítica para maximizar el rendimiento de la base de datos PostgreSQL.
- **Control de Excepciones:** Implementar un `@ControllerAdvice` global para capturar errores de serialización o fallos en los timeouts de conexión a base de datos.